CHATGPT PROMPT ENGINEERING FOR DEVELOPERS

Detailed Notes: Introduction Lecture

Course	ChatGPT Prompt Engineering for Developers
Main focus	Using instruction-tuned LLMs and APIs to build software applications
Core principles	Be clear and specific; give the model time to think

1. Course Introduction

Purpose of the course

- Teach developers to use large language models through APIs to build real applications.
- Move beyond one-off tasks in the ChatGPT web interface toward repeatable software workflows.
- Explain practical prompt-writing techniques and common LLM use cases.
- Demonstrate how to build a conversational chatbot using an LLM.

Instructors

- Andrew Ng introduces the course from the perspective of AI education, application development, and startup experience.
- Isa Fulford is introduced as an OpenAI technical staff member who worked on the ChatGPT Retrieval Plugin, contributed to the OpenAI Cookbook, and teaches developers how to use LLM technology in products.

2. Why LLMs Matter for Developers

Current use versus larger opportunity

- Many people use ChatGPT for isolated tasks such as drafting, summarizing, brainstorming, and answering questions.
- The larger, still underappreciated opportunity is to call LLMs through APIs and integrate them into software products.
- LLM APIs can shorten development cycles and help teams prototype intelligent features quickly.

Practical observation

- Andrew Ng notes that AI Fund has worked with startups applying LLM technology to many applications.
- These projects show that developers can rapidly build useful capabilities by combining prompts, application logic, and API calls.

3. Course Roadmap

Major topics

- Prompting best practices for software development.
- Summarizing: reducing long text into concise, useful outputs.
- Inferring: detecting sentiment, intent, topics, or structured information.
- Transforming: translating, rewriting, reformatting, or changing tone.
- Expanding: producing longer content such as emails or descriptions from short inputs.
- Building a chatbot: managing conversational messages and context with an LLM.

4. Two Broad Types of LLMs

Base LLMs

- A base LLM is trained mainly to predict the next word or token based on preceding text.
- It learns statistical patterns from very large collections of text, including internet and other sources.
- Its natural behavior is text continuation, not necessarily instruction following.

Examples

- For “Once upon a time there was a unicorn,” a base model may continue a story using patterns seen in training data.
- For “What is the capital of France?”, it may continue with more quiz questions because lists of questions commonly occur online. This reflects the continuation objective, not necessarily lack of knowledge.

Instruction-tuned LLMs

- An instruction-tuned LLM is trained to understand and follow user instructions.
- For the France question, it is more likely to answer directly: “The capital of France is Paris.”
- Instruction tuning aligns model behavior with the user’s requested task.

5. How Instruction-Tuned LLMs Are Developed

Stage 1: pretraining

- Train a base model on extensive text so it learns language patterns, concepts, associations, and next-token prediction.

Stage 2: supervised instruction fine-tuning

- Further train the model on pairs of instructions and high-quality responses.
- Examples teach that questions should be answered, summaries should be concise, and requested formats should be followed.

Stage 3: RLHF

- RLHF means Reinforcement Learning from Human Feedback.
- Human evaluators compare or rank candidate responses.
- The model is refined to prefer responses judged more helpful, accurate, safe, and aligned with the instruction.

Desired behavior

- Instruction-tuned models aim to be helpful, honest, and harmless.
- They are generally less likely than raw base models to produce toxic or problematic outputs.

6. Why Instruction-Tuned Models Are Recommended

Advantages

- They follow questions and commands more reliably.
- They are easier for developers to use in practical applications.
- They better reflect human intent and expected task behavior.
- Safety and alignment training can reduce harmful outputs.

Industry direction

- Research and practice have increasingly shifted toward instruction-tuned models.
- Some older prompting advice was designed for base-model continuation.
- For most current applications, developers should focus on instruction-tuned models.

7. The Right Mental Model for Prompting

Treat the LLM like a smart new colleague

- Imagine giving a task to an intelligent person who does not know the specific context of your work.
- Intelligence does not replace missing requirements. The model needs the goal, background, constraints, and expected output.
- When results are weak, the issue may be unclear instructions rather than model capability.

Communication, not magic words

- Good prompt engineering is mainly precise task communication.
- The goal is to remove ambiguity and provide usable context, not to discover a secret phrase.

8. Be Clear and Specific

Weak example

- “Please write me something about Alan Turing.” This does not define whether to discuss scientific work, personal life, historical role, or another topic.

Improved example

- “Write a 500-word professional article for undergraduate students explaining Alan Turing’s contributions to computer science and historical significance.”
- This defines topic, scope, audience, length, and tone.

Useful details to specify

- Task or objective.
- Relevant context or source material.
- Target audience.
- Scope and emphasis.
- Tone and writing style.
- Output format.
- Length or level of detail.
- Constraints, exclusions, and success criteria.

9. Specify Tone, Audience, and Format

Tone

- Possible tones include professional, academic, journalistic, casual, friendly, persuasive, or concise.

Audience

- The same subject should be explained differently to a child, undergraduate, executive, technical specialist, or customer. Audience determines vocabulary, examples, depth, and assumed knowledge.

Format

- Request bullets, a table, JSON, an email, a report, a checklist, or another structure when the output will be used downstream.

10. Provide Context and Reference Material

Why context matters

- A capable assistant performs better when given the background needed for the task.
- Supplying relevant excerpts, data, examples, or policies grounds the response in the material that matters.

Developer implication

- Applications can insert retrieved documents, product details, customer messages, or structured records into prompts.
- The model can then summarize, classify, transform, or respond using that supplied context.

11. Two Core Prompting Principles

Principle 1: write clear and specific instructions

- State exactly what the model should do.
- Define output expectations, constraints, audience, and tone.
- Provide relevant context rather than assuming the model knows the situation.

Principle 2: give the model time to think

- For tasks requiring reasoning, ask for intermediate analysis or a sequence of steps before the conclusion.
- Breaking a problem into stages can improve reliability and reduce rushed errors.
- This principle is introduced here and developed further in the next lesson.

12. Key Takeaways

Summary

- LLM APIs enable developers to build software, not merely complete isolated chat tasks.
- Base LLMs predict likely continuations; instruction-tuned LLMs are trained to follow requests.
- Instruction tuning and RLHF improve usefulness, alignment, and safety.
- Treat a model like a smart assistant that still needs context and precise requirements.
- Strong prompts identify the objective, audience, tone, format, constraints, and relevant source material.
- The two central principles are: be clear and specific, and give the model time to think.

13. Quick Reference: Weak vs. Strong Prompt

Element	Weak instruction	Stronger instruction
Topic	Write about Alan Turing.	Explain his computer-science contributions and historical impact.
Audience	Not stated	Write for undergraduate students.
Tone	Not stated	Use a professional, educational tone.
Format	Not stated	Use headings and a five-point summary.
Context	No references	Use the supplied source and avoid unsupported claims.
Reasoning	Give the answer.	Analyze requirements step by step, then answer.

14. Prompt-Writing Checklist

1. Define the task clearly.
2. Provide necessary context or source text.
3. Identify the audience.
4. Specify tone and style.
5. State the output format.
6. Add length, scope, and other constraints.
7. Break complex reasoning into steps.
8. Review the result and refine the prompt.